

레포지토리 연동 방법

07/01/2026 11:49:54

혹시 레포지토리 연동을 메모리 상에서 할 수 있는지 궁금합니다....

레포지토리를 “기억에 영구 연결”하는 것과 “요청 시점에 GitHub에서 최신 상태를 읽어 ...



혹시 레포지토리 연동을 메모리 상에서 할 수 있는지 궁금합니다.

<https://github.com/Trend-Leader-capstoneProject/feature>

해당 사이트에서 반영된 내용을 지속적으로 모니터링해서 코드 구현하려고요

2026. 7. 1. 오전 11:47:03



레포지토리를 “기억에 영구 연결”하는 것과 “요청 시점에 GitHub에서 최신 상태를 읽어 구현에 반영”하는 것은 다르게 봐야 합니다. 먼저 현재 제가 할 수 있는 연동 방식 기준으로 정리해드리겠습니다.

Thinking

● GitHub 저장소 연동 가능성 설명

사용자에게 메모리를 통한 GitHub 저장소의 지속적인 모니터링은 지원되지 않음을 설명해야 해요. 메모리는 실시간 동기화가 불가능하고, GitHub 연결기를 사용하거나 파일을 업로드하고 확인할 수 있어요. 자동화 도구를 이용해 주기적으로 상태를 확인하거나 알림을 받을 수 있는 옵션은 있죠.

● GitHub 커밋 및 파일 출처 언급

저장소 접근은 확인했으며 프로젝트 목표나 API 설정에 대해 파일에서 인용할 수 있습니다. 하지만 GitHub 도구를 사용할 때는 별도의 인용이 필요 없을 수 있어요. 사실 "방금 확인해보니"로 커넥터 원본을 언급할 수 있겠네요.

● 메모리와 레포지토리 연결 방식 설명

메모리에서는 레포지토리를 자동으로 동기화하거나 감시할 수 없어요. 메모리는 프로젝트 URL, 브랜치 정책, 아키텍처 정보를 기억할 수 있지만, 전체 코드를 저장하거나 변경 사항을 지속적으로 추적하진 않아요. 대신 세 가지 방법을 제안할 수 있습니다: 1) 확인 원하는 순간 동기화, 2) 자동화로 정해진 시간에 감시, 3) 로컬에서 작업하는 방법입니다.

✓ 28s 동안 생각함

가능합니다. 다만 “메모리 상에서 레포지토리를 실시간으로 계속 동기화해두는 것”은 불가능하고, 대신 아래 방식으로 운영하는 게 현실적입니다.

1. 가능한 것과 불가능한 것

가능한 것

제가 현재 GitHub 연결을 통해 이 레포지토리 자체는 접근할 수 있습니다.

확인된 기준으로는 기본 브랜치가 `main` 이고, 레포지토리 읽기/쓰기 권한도 연결되어 있습니다. 따라서 이후 요청하실 때 다음 작업이 가능합니다.

- 최신 커밋 / 브랜치 / PR 확인
- 특정 파일 읽기
- 변경된 코드 비교
- 구현 방향 설계
- 코드 수정안 작성
- 명시적으로 요청하면 파일 생성/수정
- 브랜치 생성, PR 생성 같은 GitHub 작업 보조

불가능한 것

반대로 이런 방식은 어렵습니다.

- 제가 레포지토리를 **항상 백그라운드에서 실시간 감시**
- GitHub 변경사항을 **자동으로 메모리에 계속 반영**
- 사용자 확인 없이 코드 자동 수정/커밋
- 모든 소스코드를 장기 기억처럼 저장해두고 매번 자동 참조

즉, 메모리는 프로젝트 방향성, 선호 구조, 이전 결정사항을 기억하는 용도에 가깝고, 실제 최신 코드는 매번 **GitHub에서 다시 읽어 확인하는 방식**이 안전합니다.

2. 가장 좋은 작업 방식

현재 Trend Leader는 관심사 기반 개인화 트렌드 큐레이션 앱이고, 핵심 기능은 관심사 선택, 관심사 기반 추천, 트렌드 상세 조회, AI 요약 중심으로 잡혀 있습니다. `interest_based_trend_app_plan.md`

API 구성도도 이미 꽤 정리되어 있습니다. 기술 기준은 React Native, FastAPI, MySQL/MariaDB, JWT Bearer Token, 공통 응답 구조 `success`, `statusCode`, `message`, `data` 기준으로 잡혀 있습니다. `Trend Leader API 구성표.pdf`

그래서 앞으로는 이렇게 진행하는 게 좋습니다.

3. 추천 워크플로우

1단계: GitHub 최신 상태 확인

요청 예시는 이렇게 주시면 됩니다.

Trend-Leader feature 레포지토리 최신 main 기준으로 backend 구조를 확인하고, 다음에 구현할 API 우선순위를 잡아주세요.

또는

현재 레포지토리에서 Interest API 구현 상태를 확인하고, 카테고리 목록 조회 API부터 구현 방향을 잡아주세요.

이러면 제가 GitHub에서 최신 파일을 읽고, 현재 구조를 기준으로 분석할 수 있습니다.

2단계: 문서 기준으로 구현 우선순위 잡기

현재 API 작성 우선순위는 다음 흐름이 적절합니다.

1. 카테고리 목록 조회
2. 관심사 저장
3. 기존 관심사 조회
4. 관심사 수정
5. 맞춤 트렌드 목록 조회
6. 전체 트렌드 목록 조회
7. 트렌드 상세 조회
8. 북마크
9. 검색

이 우선순위는 기존 API 구성표에서도 Interest API를 먼저 작성 대상으로 잡고, 이후 Trend / Bookmark / Search API로 확장하는 흐름과 일치합니다. [Trend Leader API 구성표.pdf](#)

3단계: 구현 단위는 “API 하나씩” 가는 게 좋습니다

예를 들어 지금 가장 먼저 구현하기 좋은 것은 이것입니다.

GET /api/categories

이 API는 관심사 선택 화면을 렌더링하기 위한 선행 API라서, 프론트와 백엔드 연결의 첫 기준으로 좋습니다. API 구성표에서도 카테고리 목록 조회는 관심사 선택 화면과 관심사 재설정 화면에서 사용하는 API로 정리되어 있습니다. `Trend Leader API 구성표.pdf`

DB 기준으로도 `categories` 테이블이 이미 존재하고, `category_id`, `parent_id`, `category_name`, `depth`, `sort_order`, `is_active` 필드를 가지고 있습니다. `Trend_Leader_fixed.sql`

4. 제가 앞으로 도와드릴 수 있는 실제 방식

앞으로는 이렇게 요청하시면 됩니다.

레포지토리 최신 상태 기준으로 GET /api/categories 구현을 시작해주세요.

단, 바로 코드 생성하지 말고 현재 구조 분석 → 설계안 → 수정 파일 목록 → 코드 순서로 진행해주세요.

그러면 제가 보통 이런 순서로 진행합니다.

1. GitHub 레포지토리 최신 구조 확인
2. 현재 FastAPI 라우터 / 서비스 / DB 연결 구조 파악
3. 기존 코드 스타일 확인
4. `categories` API 구현 위치 결정
5. 필요한 파일 목록 제시
6. 코드 수정안 제시
7. 테스트 방법 안내
8. 원하시면 GitHub에 파일 수정까지 진행

5. “지속 모니터링”에 가까운 대안

완전한 실시간 감시는 아니지만, 자동 확인용으로는 이런 형태가 가능합니다.

매일 오후 9시에 Trend-Leader feature 레포지토리의 최신 커밋과 PR 변경사항을 확인해서 알려줘.

이런 식의 예약 확인은 만들 수 있습니다. 다만 이 경우에도 **제가 몰래 코드를 구현해서 커밋하는 방식은 아니고**, 변경사항을 요약해드리고 사용자가 “이제 구현하자”라고 요청하면 작업하는 방식이 안전합니다.

결론

정리하면, 레포지토리를 메모리에 실시간 연결해두는 것은 불가능하지만, **GitHub 레포지토리를 요청 시점마다 최신 상태로 읽고, 그 기준으로 설계·구현·수정·PR 작업을 이어가는 것은 가능합니다.**

현재 프로젝트라면 가장 좋은 시작점은 다음입니다.

Trend-Leader feature 레포지토리 최신 main 기준으로 backend 구조를 확인하고,
GET /api/categories 카테고리 목록 조회 API 구현 설계부터 잡아주세요.

이 흐름으로 가면 “바이브 코딩”처럼 즉흥 구현하지 않고, 문서 → DB → API → 코드 구조 순서로 안정적으로 진행할 수 있습니다.

2026. 7. 1. 오전 11:47:04